SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

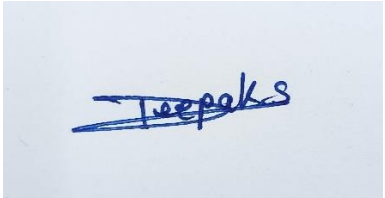
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I S Deepak_1921260077 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Q1. ATM Withdrawal Module: Reverse Engineering Black-Box Test Design

10 Marks

1. Given Artefacts and Observed Behaviour

Only the input-output behaviour of the ATM withdrawal module is available. The observations below are an **assumed sample** of the partially tested module (balance, daily total already withdrawn, requested amount and system response).

#	Balance	Today so far	Request	Observed response
1	20,000	0	500	Cash dispensed
2	20,000	0	450	Enter amount in multiples of 100
3	20,000	0	0 / -500 / abc	Invalid amount
4	20,000	0	100	Cash dispensed
5	20,000	0	10,000	Cash dispensed
6	20,000	0	10,100	Exceeds per-transaction limit
7	800	0	900	Insufficient balance
8	800	0	800	Cash dispensed, balance 0
9	30,000	45,000	5,000 / 5,100	Dispensed / Daily limit exceeded
10	800	0	10,100	Exceeds per-transaction limit (checked before balance)

Table 1: Observed input-output pairs (assumed sample)

2. Decomposition into Functional Components

Component	Responsibility	Evidence in observations
Input validator	Numeric, positive, non-zero amount	Rows 3
Denomination checker	Amount must be a multiple of 100	Row 2
Per-transaction limit	Maximum 10,000 per withdrawal	Rows 5, 6, 10
Daily limit tracker	Maximum 50,000 per day	Row 9
Balance validator	Amount must not exceed balance	Rows 7, 8
Dispenser and ledger update	Dispense cash, reduce balance	Rows 1, 8

The order of the error messages shows the **processing sequence**: format, then denomination, then per-transaction limit, then daily limit, then balance (row 10).

3. Inferred Equivalence Classes and Boundaries

Input	Valid class	Invalid classes	Boundary values
Amount	100 to 10,000, multiple of 100	Zero or negative; non-numeric; not a multiple of 100; above 10,000	0, 99, 100, 101, 9,900, 10,000, 10,100
Amount vs balance	Amount <= balance	Amount > balance	balance-1, balance, balance+1

Input	Valid class	Invalid classes	Boundary values
Daily total	today + amount <= 50,000	today + amount > 50,000	49,900 / 50,000 / 50,100

4. Techniques Inferred and Adequacy Evaluation

The pattern of the observations shows that the original testers used **Equivalence Class Partitioning** (one value per class: 450, 0, 900) and **Boundary Value Analysis** on only two limits (10,000 / 10,100 and balance equals amount).

Area	Covered?	Gap
Valid class and per-transaction maximum	Yes	None
Minimum boundary (99, 100, 101)	Partly	Only 100 tried; 99 and 0 < x < 100 missing
Daily limit boundary	Partly	Combined with balance and limit not tried
Non-numeric, decimal (100.50), blank input	No	Not observed
Multiple failures at once	Row 10 only	Priority of other checks untested

5. Reconstructed Test Cases

TC	Balance / Today	Input	Technique	Expected result
T1	20,000 / 0	99	BVA (min-1)	Rejected: not a valid denomination
T2	20,000 / 0	100	BVA (min)	Dispensed
T3	20,000 / 0	200	BVA (min+1)	Dispensed
T4	20,000 / 0	0 and -100	ECP invalid	Invalid amount
T5	20,000 / 0	abc, blank, 100.50	ECP invalid	Invalid amount
T6	20,000 / 0	350	ECP invalid	Multiples of 100 message
T7	20,000 / 0	9,900	BVA (max-1)	Dispensed
T8	20,000 / 0	10,000	BVA (max)	Dispensed
T9	20,000 / 0	10,100	BVA (max+1)	Per-transaction limit error
T10	800 / 0	700 / 800 / 900	BVA on balance	Dispensed / Dispensed / Insufficient
T11	30,000 / 45,000	5,000	BVA daily	Dispensed (total = 50,000)
T12	30,000 / 45,000	5,100	BVA daily	Daily limit exceeded
T13	500 / 49,900	600	Combination	Daily limit error first, then balance

Key Insight

The behaviour reveals both ECP and BVA, but only at the upper limits. The reconstructed suite completes the lower boundaries, input-format classes and the priority of overlapping rules.

6. Justification

- **Critical coverage:** every inferred rule has at least one valid and one invalid test with boundary values on both sides.
- **Efficiency:** 13 cases replace exhaustive testing of thousands of possible amounts.

- **Defect detection:** off-by-one errors such as `<` instead of `<=` appear exactly at the boundary values T1, T3, T8, T9, T11 and T12.

Q2. Reverse Engineering White-Box Testing from Code and Execution Traces

10 Marks

1. Given Artefacts

A module snippet and three execution traces are given; no test documentation exists. The snippet below is an **assumed sample** (numbers are statement ids used in the CFG).

```
1  read(n, a[], limit)
2  if (n < 0) {
3      return ERROR }
4  total = 0; bad = 0; i = 1
5  while (i <= n) {
6      if (a[i] < 0)
7          bad = bad + 1
8      else total = total + a[i]
9      i = i + 1 }
10 if (total > limit)
11     total = limit
12 return total
```

```
T1: n=3, a=[10,20,30], limit=100  path 1,2,4,(5,6,8,9)x3,5,10,12,13
T2: n=-1                          path 1,2,3,13
T3: n=2, a=[60,70], limit=100     path 1,2,4,(5,6,8,9)x2,5,10,11,12,13
```

2. Control Flow Graph (Reconstructed)

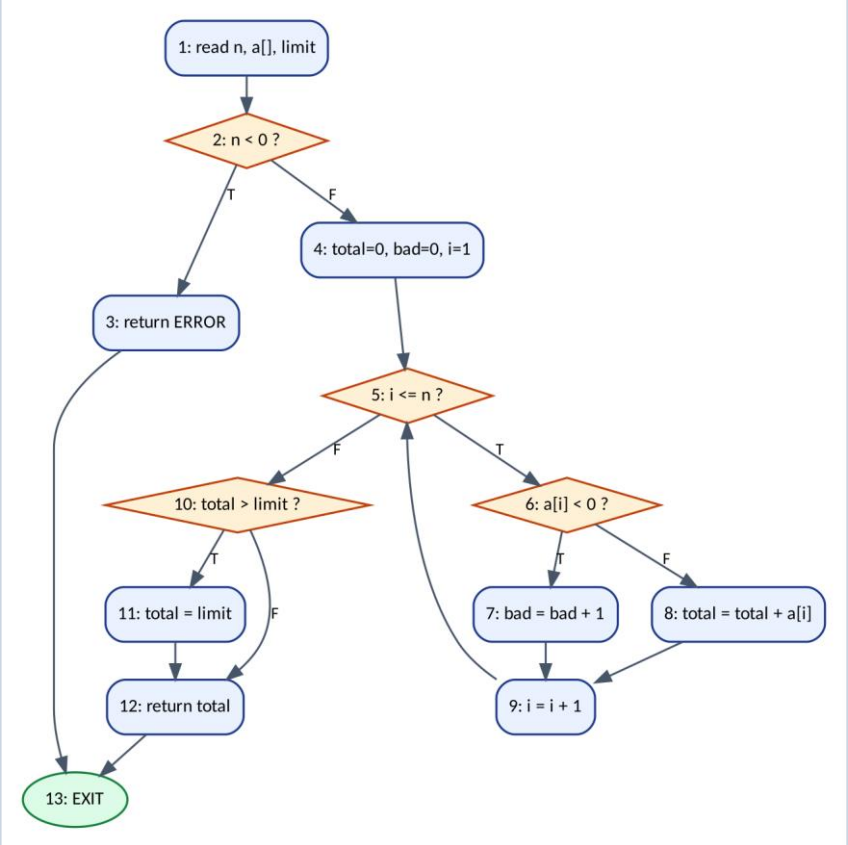


Figure 1: CFG of the module (decision nodes shown as diamonds)

3. Decision Points, Loops and Cyclomatic Complexity

Item	Nodes	Count
Decision points	2 (n<0), 5 (loop condition), 6 (a[i]<0), 10 (total>limit)	4
Loops	5-6-(7|8)-9-5	1
Nodes N / Edges E	N = 13, E = 16	
V(G) = E - N + 2	16 - 13 + 2	5
V(G) = decisions + 1	4 + 1	5

4. Independent Paths (Basis Set)

Path	Node sequence	Test input that drives it
P1	1-2-3-13	n = -1
P2	1-2-4-5-10-12-13	n = 0, limit = 100
P3	1-2-4-5-6-8-9-5-10-12-13	n = 1, a = [10], limit = 100
P4	1-2-4-5-6-7-9-5-10-12-13	n = 1, a = [-5], limit = 100
P5	1-2-4-5-6-8-9-5-10-11-12-13	n = 1, a = [150], limit = 100

Note on infeasible paths

The structural path 4-5-10-11 (zero iterations with total > limit) cannot be executed when limit >= 0, because total stays 0. Such paths are counted in V(G) but must be documented as infeasible, so P5 uses one iteration instead.

5. Evaluation of Existing Test Adequacy

Criterion	Covered by T1-T3	Result
Statement coverage	11 of 12 statements (statement 7 never run)	91.7%
Branch coverage	7 of 8 outcomes (6-True missing)	87.5%
Path coverage (basis)	P1, P3, P5 only (as P3 and P5 equivalents)	3 of 5 = 60%
Loop testing	Only 2 and 3 iterations	0, 1 and max untested

The traces show that the original approach was **functional happy-path testing with a little error testing**: no negative element, no empty loop.

6. Reconstructed Path Testing Strategy

- **Basis path tests** P1-P5 above, giving 100% statement and branch coverage.
- **Loop tests:** n = 0 (skip), 1 (once), 2 (typical), n_max - 1, n_max, n_max + 1.
- **Data-flow checks:** definition of `total` at 4/8/11 and use at 10/12, verified by tests P3, P5.

Conclusion

Five independent paths plus loop-boundary tests give full coverage. The existing traces reached about 60 to 90% depending on the criterion, which is why the defect in the negative-element branch remained hidden.

Q3. Web Login and Transaction System: Reverse Engineering the Testing Approach

10 Marks

1. Given Artefacts: Defect Log and Test Results

Only defect logs and results exist. The summary below is an **assumed sample**: 142 tests executed, 139 passed; all passing tests used mid-range values (password length 10, amount 5,000).

ID	Defect observed	Failing input
D-101	Password of 7 characters accepted	length 7
D-102	Fourth wrong attempt still allowed	attempt 4
D-103	Transfer of 0.00 succeeds	amount 0.00
D-104	Transfer of exactly 100,000.00 rejected	amount = limit
D-105	Back button after logout shows account page	logout then back
D-106	Double click creates duplicate transaction	two requests
D-107	Apostrophe in user name causes server error	o'brien
D-108	Balance becomes negative on concurrent debit	two debits

2. Decomposition into Testable Components



Figure 1: Functional decomposition of the web system

Component	Testable behaviour	Technique suited
Login UI and Auth	Credential format, lockout counter	ECP, BVA
Session manager	Creation, timeout, invalidation on logout	State/path testing
Transaction service	Amount rules, duplicate request, funds check	BVA, branch and path
Ledger DB	Atomic debit and credit	Concurrency path tests

3. Inference of Techniques Used

Evidence	Inference
All passing tests use mid-range values	ECP with one representative per valid class; no BVA
D-101 to D-104 fail exactly at limits	Boundary Value Analysis not applied
D-107 fails on special characters	Invalid ECP classes (injection, special characters) not designed
D-105, D-106, D-108 fail on sequences	Only happy-path and statement coverage; branches and paths untested

4. Gaps in Coverage

The suite reached high statement coverage (every line ran once) but low **branch coverage** of the error branches (session expired, duplicate request, insufficient funds under concurrency) and no boundary coverage.

5. Reconstructed Comprehensive Strategy

Field	ECP classes	BVA test values
Password length (8 to 16)	<8 invalid, 8-16 valid, >16 invalid	7, 8, 9, 15, 16, 17
Failed attempts (lock at 3)	0-2 allowed, 3 locked	2, 3, 4
Amount (0.01 to 100,000.00)	<=0, valid, >limit	0.00, 0.01, 0.02, 99,999.99, 100,000.00, 100,000.01
User name	Valid, empty, special characters, too long	'', a, o'brien, 51 chars

Path testing of the transaction function (decisions: amount valid, session active, funds sufficient, request id new; V(G) = 5):

Path	Scenario	Expected
P1	Amount invalid	Reject with message
P2	Amount valid, session expired	Redirect to login
P3	Valid, session active, funds insufficient	Reject, no debit
P4	Valid, funds ok, duplicate request id	Ignore duplicate
P5	All conditions true	Debit and credit committed

6. Justification

Each defect in the log maps to a technique now present in the strategy: BVA catches D-101 to D-104, extra invalid ECP classes catch D-107, and path tests for session, duplicate and concurrency branches catch D-105, D-106 and D-108. The combined strategy is therefore logically complete with respect to the observed defect patterns.

Q4. Legacy Inventory System: Inferring Hidden Boundaries from Transaction Logs

10 Marks

1. Given Artefacts: Transaction Logs

The logs below are an **assumed sample** for SKU-114.

```
RECEIVE qty=50      stock 120 -> 170   OK
SALE    qty=30      stock 170 -> 140   OK
SALE    qty=20      stock  30 ->  10   OK  (no alert)
SALE    qty=1       stock  10 ->   9   OK  ALERT=LOW_STOCK
SALE    qty=140     stock 140 ->   0   OK  ALERT=LOW_STOCK
SALE    qty=1       stock   0          FAIL E_NEG_STOCK
RECEIVE qty=0                          FAIL E_QTY_INVALID
RECEIVE qty=9999    stock   1 -> 10000 OK
RECEIVE qty=10000                      FAIL E_QTY_MAX
RECEIVE qty=4999    stock 95000 -> 99999  OK
RECEIVE qty=5000    stock 95000        FAIL E_CAPACITY
```

2. Inferred Business Rules and Hidden Boundaries

Rule	Log evidence	Boundary
Quantity must be at least 1	qty=0 fails	0 invalid, 1 valid (assumed)
Maximum 9,999 units per update	9999 OK, 10000 fails	9,999 / 10,000
Stock cannot go negative	Sale of 1 at stock 0 fails	Result 0 valid, -1 invalid
Low-stock alert when stock < 10	10 no alert, 9 alert	10 / 9
Storage capacity 99,999	99,999 OK, 100,000 fails	99,999 / 100,000

3. Decomposition into Functional Units

- **Quantity validator** (range 1 to 9,999 and integer).
- **Transaction dispatcher** (RECEIVE adds, SALE subtracts).
- **Stock guard** (result must lie between 0 and 99,999).
- **Alert evaluator** (stock after update < 10).
- **Audit writer** (success or error code logged).

4. Equivalence Partitions Inferred

Variable	Valid partitions	Invalid partitions
Quantity	1 to 9,999	<= 0; >= 10,000; decimal; non-numeric
Stock after update	0 to 9 (alert); 10 to 99,999 (no alert)	< 0; > 99,999
Transaction type	RECEIVE, SALE	Unknown type

5. Reconstructed Black-Box Test Design (Gaps in the Logs)

TC	Type / stock / qty	Expected	Gap closed
T1	RECEIVE, 100, 1	OK, 101	Lower quantity boundary
T2	RECEIVE, 100, -1	E_QTY_INVALID	Negative quantity
T3	RECEIVE, 100, 2.5	E_QTY_INVALID	Decimal class
T4	SALE, 20, 11	OK, 9, ALERT	Alert from above 10 directly
T5	SALE, 20, 10	OK, 10, no alert	Exact threshold
T6	SALE, 5, 6	E_NEG_STOCK	Negative result
T7	SALE, 5, 5	OK, 0, ALERT	Zero stock
T8	RECEIVE, 0, 9999	OK, 9999	Max quantity from empty
T9	RECEIVE, 99999, 1	E_CAPACITY	Capacity edge
T10	ADJUST, 10, 5	Error: unknown type	Invalid type
T11	SALE, 9, 1 (repeat alert)	OK, 8, ALERT once or again?	Alert repetition rule

6. Justification

The test cases follow directly from the inferred logic: each boundary (1, 9,999, 0, 10, 99,999) is tested on both sides, and each inferred rule has at least one pass and one fail case. Cases T3, T10 and T11 deliberately target conditions the logs never reveal, because logs only describe what was tried, not what was left out.

> **Key Insight**
> Reverse engineering from logs yields the likely rules, which must be confirmed with the business owner (for example, whether the alert repeats). Unconfirmed rules such as T11 are tested as open questions.

Q5. Library Management .exe: Mapping Resource Profile to Loops and Decisions

10 Marks

1. Given Artefacts

Only the compiled executable and a CPU/memory profile from a stress test exist. The profile below is an **assumed sample** (time in seconds).

Time window	CPU %	Memory (MB)	Observation
0 - 2	8	40	Flat; start-up and configuration
2 - 6	35 to 90 (rising)	40 to 220 (linear)	Records loaded progressively
6 - 9	90 with spikes every ~0.5 s	220 (flat)	Repeated bursts per batch of records
9 - 10	55 in irregular short bursts	220	Work done only for some records
10 - 12	30 falling to 5	220 down to 60	Report writing, then release

2. Mapping Execution Spikes to Code Structures

Profile feature	Likely code structure	Reason
Linear memory growth, rising CPU	Outer loop over members	Work and memory grow with record count
Periodic spikes at fixed spacing	Inner loop over each member's loans	One spike per outer iteration (nested loop)
Short irregular bursts	Decision cluster (overdue / reserved)	Extra work only when a condition is true
Memory returned at the end	Report written, then objects released	No leak at the exit node

3. Conceptual Control Flow Graph

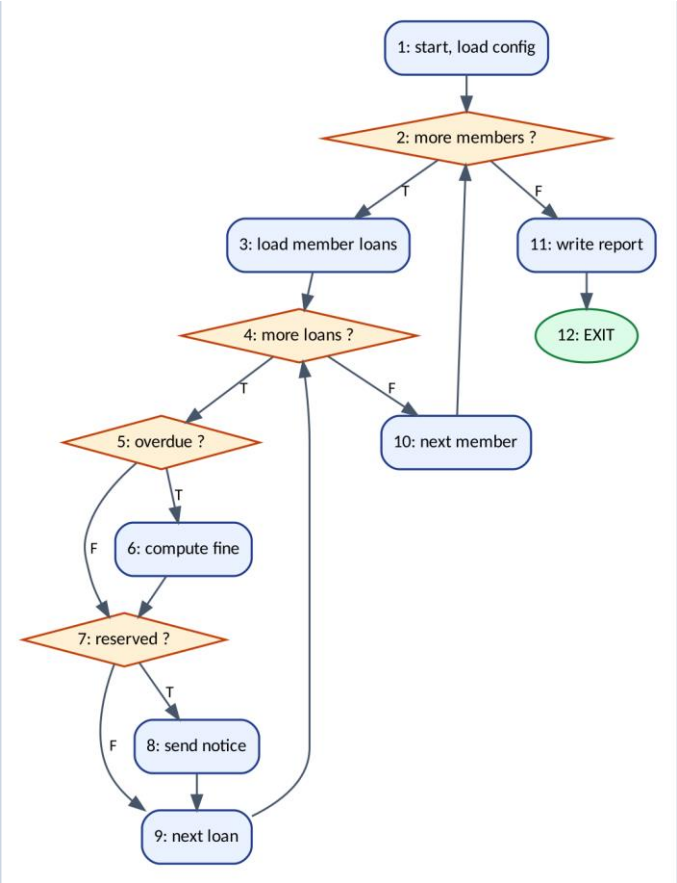


Figure 1: Conceptual CFG reconstructed from the profile

N = 12, E = 15, so V(G) = 15 - 12 + 2 = **5**, which equals decisions (nodes 2, 4, 5, 7) + 1.

4. Reconstructed Path Testing Strategy

Path	Node sequence	Test scenario
P1	1-2-11-12	No members
P2	1-2-3-4-10-2-11-12	One member, zero loans
P3	1-2-3-4-5-7-9-4-10-2-11-12	Loan neither overdue nor reserved
P4	1-2-3-4-5-6-7-9-4-10-2-11-12	Overdue loan, fine computed
P5	1-2-3-4-5-7-8-9-4-10-2-11-12	Reserved loan, notice sent
P6	Overdue and reserved (5-6-7-8)	Heaviest iteration, supplementary

P6 is the most resource-intensive path and is tested in addition to the basis set.

Stress test	Settings	Pass criteria
Loop boundary	Members: 0, 1, 5,000, 5,001	No crash; correct count
Inner loop	Loans per member: 0, 1, 10, 11	Correct fines
Sustained load	5,000 members x 10 loans, all overdue and reserved	CPU peak < 95%, memory flat at 220 MB or less
Repeat runs	20 consecutive runs	Memory returns to about 40 MB each time

5. Impact on System Stability

- **Hot loops located:** testing at loop limits exposes timeouts and memory exhaustion before production.
- **Leak detection:** the repeat-run test verifies that memory after node 12 returns to baseline.
- **Priority:** the heaviest path P6 is verified first because it determines the worst-case capacity.

Conclusion

Mapping resource spikes to loops and decision clusters gives a conceptual CFG with five basis paths. The reconstructed strategy targets those paths and loop limits, improving both correctness and stability.

Q6. Payment Gateway API: Reconstructing the State Machine

10 Marks

1. Given Artefacts: API Responses

The JSON below is an **assumed sample** of gateway responses.

```
{"txn_id":"T1001","state":"PENDING","amount":2500}
{"txn_id":"T1001","state":"SUCCESS","bank_ref":"BK98213"}
{"txn_id":"T1002","state":"PENDING"}
{"txn_id":"T1002","state":"TIMEOUT","reason":"NO_ACK_30S","retry_count":0}
{"txn_id":"T1002","state":"PENDING","retry_count":1}
{"txn_id":"T1003","state":"SUCCESS"} -> {"state":"REVERSAL","reason":"CUSTOMER_REFUND"}
{"txn_id":"T1004","state":"REVERSAL"} + "SUCCESS" request -> {"error":"E409_INVALID_TRANSITION"}
```

2. Analysis of Transition Triggers and Data Constraints

Observation	Inferred trigger or constraint
PENDING to SUCCESS carries bank_ref	Trigger: bank acknowledgement; bank_ref mandatory
PENDING to TIMEOUT has NO_ACK_30S	Trigger: 30 second timer
TIMEOUT to PENDING has retry_count	Retry allowed, counter limited (assumed 3)
SUCCESS to REVERSAL has reason	Refund allowed, reason mandatory, within 24 hours (assumed)
REVERSAL to SUCCESS gives 409 error	REVERSAL is a terminal state

3. Reconstructed State Machine and Technique Identified

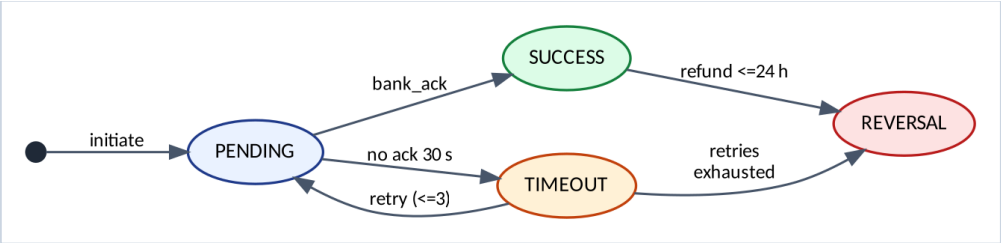


Figure 1: State-transition diagram (PENDING is the entry state)

The implicit technique is **State Transition Testing**: states, events, guard conditions (retry count, 24-hour window) and rejection of illegal transitions.

4. Legal and Illegal Transition Table (All 12 Pairs)

ID	From	To	Type	Expected
TT1	PENDING	SUCCESS	Legal	State changes, 200 OK
TT2	PENDING	REVERSAL	Illegal	Rejected, error E409_INVALID_TRANSITION
TT3	PENDING	TIMEOUT	Legal	State changes, 200 OK
TT4	SUCCESS	PENDING	Illegal	Rejected, error E409_INVALID_TRANSITION
TT5	SUCCESS	REVERSAL	Legal	State changes, 200 OK
TT6	SUCCESS	TIMEOUT	Illegal	Rejected, error E409_INVALID_TRANSITION

ID	From	To	Type	Expected
TT7	REVERSAL	PENDING	Illegal	Rejected, error E409_INVALID_TRANSITION
TT8	REVERSAL	SUCCESS	Illegal	Rejected, error E409_INVALID_TRANSITION
TT9	REVERSAL	TIMEOUT	Illegal	Rejected, error E409_INVALID_TRANSITION
TT10	TIMEOUT	PENDING	Legal	State changes, 200 OK
TT11	TIMEOUT	SUCCESS	Illegal	Rejected, error E409_INVALID_TRANSITION
TT12	TIMEOUT	REVERSAL	Legal	State changes, 200 OK

5. Comprehensive State-Based Test Suite

Coverage level	Tests	Count
0-switch (each transition once)	All 5 legal transitions	5
Invalid transitions	All 7 illegal pairs, expect 409	7
1-switch sequences	PENDING-TIMEOUT-PENDING-SUCCESS; PENDING-SUCCESS-REVERSAL	2
Guard conditions	retry_count 3 allowed, 4 forces REVERSAL; refund at 24 h and 24 h 1 s; missing bank_ref	5

6. Justification

The suite covers all states, all legal transitions and all illegal transitions, so no state-pair is left untested. Because terminal states must reject further events, the illegal-transition tests are the principal protection against double payment and wrong refunds. Guard-value tests add boundary coverage that ECP or BVA on the data fields alone would not give.

Q7. Encrypted Communication Module: Fuzzing and BVA Reconstruction

10 Marks

1. Given Artefacts

Only input parameters and encrypted output are visible. The parameters are an **assumed sample**: `sender` (text, up to 32 characters), `key` (32 hex characters), `message` (text), `mode` (CBC or GCM).

2. Decomposition

Component	Function	What fuzzing reveals
Data validation	Length, character set, key format, mode	Error codes at specific sizes or characters
Encryption engine	Encrypt validated data	Output length and format; crashes on bad data

3. Fuzzing Analysis: Observed Error-Handling Boundaries

Field	Fuzz input	Response	Inference
message	256 chars	Encrypted	Limit probably 256
message	257 chars	ERR_LEN	Length check at 256
message	empty	ERR_EMPTY	Minimum 1
key	31 / 33 hex chars	ERR_KEY	Exactly 32
key	32 chars with letter G	ERR_KEY	Hex check present
sender	33 chars	ERR_SENDER	Limit 32
mode	ECB	ERR_MODE	Whitelist of CBC and GCM
message	65,536 chars	Connection reset	Missing early length check: possible overflow

4. Reconstructed Test Cases Using BVA

TC	Field	Value	Expected
T1	message	empty (0)	ERR_EMPTY
T2	message	1 char	Encrypted
T3	message	2 chars	Encrypted
T4	message	255 chars	Encrypted
T5	message	256 chars	Encrypted
T6	message	257 chars	ERR_LEN
T7	key	31, 32, 33 hex chars	ERR_KEY, OK, ERR_KEY
T8	sender	32 chars / 33 chars	OK / ERR_SENDER
T9	message	4,097 and 65,536 chars	ERR_LEN, no crash

Security-oriented fuzz inputs:

Attack class	Input	Expected behaviour
SQL injection	' OR '1'='1	Treated as plain text or rejected
Script injection	<script>alert(1)</script>	Encrypted as data, never executed
Path traversal	../../etc/passwd	Rejected or treated as text
Format string	%s%s%s%n	No memory fault
Null byte	abc, then NUL, then def	Rejected or fully handled
Buffer overflow	"A" repeated 100,000 times	ERR_LEN before copying to buffer

5. Justification

- **Overflow prevention:** BVA at 255, 256 and 257 proves that the length check fires before data is copied; the oversized inputs show no crash.
- **Injection prevention:** hostile strings must be accepted only as data or rejected, never interpreted.
- **Separation:** validation failures are distinguished from encryption failures by distinct error codes.

Conclusion

Fuzzing reveals the boundaries, BVA turns them into precise tests, and the attack inputs confirm that validation protects the encryption component.

Q8. Insurance Premium Calculator: Reconstructing Branch and Path Coverage

10 Marks

1. Given Artefacts

Legacy results show 100% statement coverage but missed defects in multi-condition decisions. The logic below is an **assumed sample** (base premium 1,200).

```
premium = base
if (age < 25 && claims > 0)      premium = premium * 1.5   // D1
if (smoker || bmi > 30)         premium = premium * 1.2   // D2
if (years >= 5)                 premium = premium * 0.9   // D3
return premium
```

2. Reverse Engineering the Original Approach

The legacy suite most likely contained **one test** (young driver with claim, smoker, 6 years). That single input executes every statement, so statement coverage is 100%, but only the True outcome of every decision is exercised.

Criterion	Achieved by one test	Reason
Statement coverage	100%	All statements 1 to 8 executed
Branch coverage	3 of 6 = 50%	No False outcome of D1, D2 or D3
Condition coverage	Incomplete	age >= 25, claims = 0, non-smoker and bmi <= 30 never tried
Path coverage	1 of 8 = 12.5%	Seven path combinations unexecuted

Why defects escaped

Statement coverage ignores the False side of each decision and ignores combinations of conditions inside `&&` and `||`, so an error such as `||` coded as `&&` stays hidden.

3. New Control Flow Graph

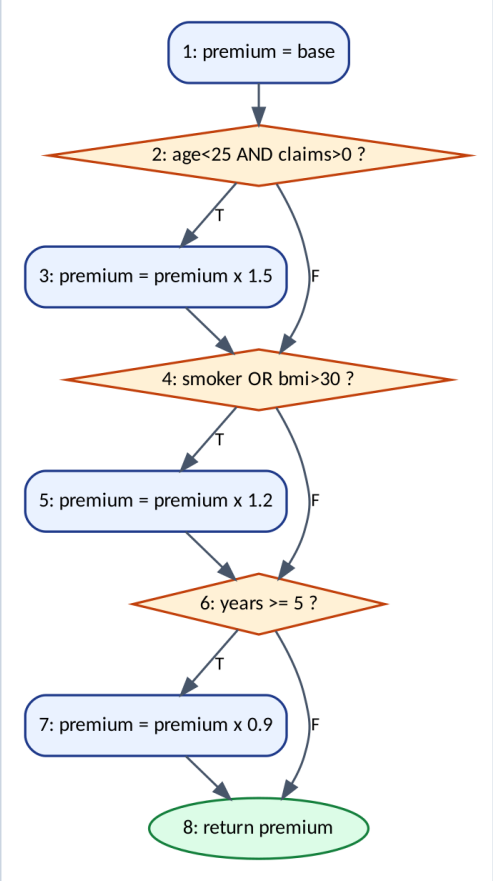


Figure 1: CFG of the premium calculator

N = 8, E = 10, so V(G) = 10 - 8 + 2 = **4**, which equals 3 decisions + 1.

4. Independent Paths (Basis Set)

Path	Nodes	Description
P1	1-2-4-6-8	Baseline: all decisions False
P2	1-2-3-4-6-8	Flip D1 only
P3	1-2-4-5-6-8	Flip D2 only
P4	1-2-4-6-7-8	Flip D3 only

5. Reconstructed Strategy for 100% Branch and Path Coverage

Because there are no loops, the three decisions give 2 x 2 x 2 = 8 feasible paths. Test data (age / claims / smoker / BMI / years):

Path (D1 D2 D3)	Inputs	Expected premium
TTT	22 / 1 / Yes / 24 / 6	1,944
TTF	22 / 1 / Yes / 24 / 2	2,160
TFT	22 / 1 / No / 24 / 6	1,620
TFF	22 / 1 / No / 24 / 2	1,800

Path (D1 D2 D3)	Inputs	Expected premium
FTT	40 / 0 / No / 33 / 6	1,296
FTF	40 / 0 / Yes / 24 / 2	1,440
FFT	40 / 0 / No / 24 / 6	1,080
FFF	40 / 0 / No / 24 / 2	1,200

Coverage target	Tests required	Selected
Branch coverage 100%	2	TTT and FFF
Basis path coverage	4	FFF, TFF, FTF, FFT
All paths 100%	8	All rows above

Condition level (MC/DC) for the compound decisions:

Decision	Tests (conditions)	Shows
age<25 AND claims>0	(T,T), (F,T), (T,F)	Each condition alone changes the outcome
smoker OR bmi>30	(F,F), (T,F), (F,T)	Each condition alone changes the outcome

6. Reduced Risk of Calculation Errors

Each multiplier is applied in isolation (basis paths) and in every combination (eight paths), so errors in order, operator or threshold such as `years > 5` instead of `years >= 5` are exposed by the years = 5 boundary. The reconstruction raises branch coverage from 50% to 100% and path coverage from 12.5% to 100%.

Q9. Cloud File Sync Tool: Inferring ECP from Lost-File Defects

10 Marks

1. Given Artefacts: Defect Patterns

The sample below is an **assumed** summary of 35 "lost file" reports.

Pattern	File size	Type	Network	Reports
A	>= 2 GB	Any	Any	14
B	100 MB to 2 GB	Media	Below 1 Mbps	9
C	0 bytes	Any	Any	3
D	Small	Name > 255 chars or special characters	Any	5
E	Small	.tmp and .lock	Any	4
F	Below 100 MB	Documents	Above 1 Mbps	0

2. Reverse Engineered Algorithm

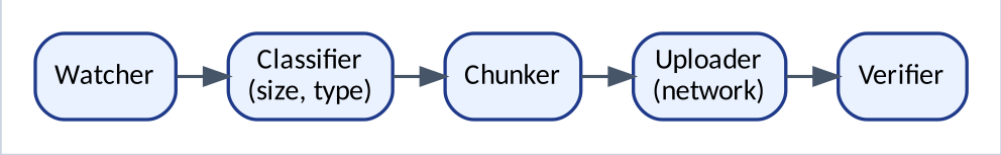


Figure 1: Inferred synchronisation pipeline

- **Pattern A:** exactly the 32-bit signed integer limit (2,147,483,647 bytes); larger files are truncated or skipped.
- **Pattern B:** large files are chunked, but the chunk upload times out on slow networks without retry.
- **Pattern C:** zero-byte files are not handled by the classifier.
- **Patterns D and E:** name validation and an exclusion list silently drop files.

3. Inferred ECP Used by the Developers

Variable	Inferred classes
Size	S0: 0 B | S1: 1 B to 100 MB (single upload) | S2: 100 MB to 2 GB (chunked) | S3: above 2 GB (unsupported)
Network	N0: offline | N1: below 1 Mbps | N2: 1 to 10 Mbps | N3: above 10 Mbps
Type	T1: documents | T2: media | T3: temporary or system

The developers evidently tested only S1 x N3 x T1 (the best case), which explains zero reports for pattern F.

4. Reconstructed Strategy: BVA Combined with Stress Testing

TC	Size / network	Expected result
T1	0 B, 10 Mbps	Synced as empty file
T2	1 B, 10 Mbps	Synced
T3	100 MB - 1 B and 100 MB + 1 B	Single upload, chunked upload; checksum equal

TC	Size / network	Expected result
T4	2,147,483,647 B and 2,147,483,648 B	Both synced or clear error, never lost
T5	1 GB at 0.9, 1.0 and 1.1 Mbps	Retry on timeout; complete upload
T6	Name with 255 and 256 chars	Synced / clear error message
T7	File with special characters and unicode	Synced

Stress test	Load	Pass criteria
Burst	10,000 small files at once	All files listed; none lost
Concurrency	50 simultaneous uploads	No overwrite or corruption
Network drop	Disconnect during chunk 3	Resume, no data loss
Disk full	Remote quota exceeded	File kept locally, error shown

5. Prevention of Future Data Loss

BVA at 100 MB and 2 GB targets the exact thresholds where defects A and B arise. Stress and network-fault tests reproduce the conditions that production users meet, and checksum verification after every sync proves that no file was silently dropped. Together they close each defect pattern in the log.

Conclusion

The defects form clear boundary and class patterns; testing every size boundary under realistic network and load conditions prevents recurrence.

Q10. E-commerce Search and Filter: Decision Table Reconstruction

10 Marks

1. Given Artefacts: SQL Sequence

The queries below are an **assumed sample** captured from the module.

```
Q1: SELECT id,name,price FROM products WHERE category='Shoes' AND price BETWEEN 50 AND 150;
Q2: ... WHERE category='Shoes' AND brand IN ('Nike','Adidas') AND in_stock=1;
Q3: ... WHERE category='Shoes' AND NOT (brand='Puma') AND in_stock=1;
Q4: ... WHERE brand IN ('Nike','Adidas') AND price <= 100;
Q5: ... WHERE 1=1;   -- no filter selected
```

2. Inferred Filtering Logic

Observation	Inferred rule
Different filters are joined by AND	A product must satisfy every selected facet
Several brands appear in IN (...)	Values inside one facet are combined by OR
NOT (brand=...) in Q3	An Exclude option is supported
Q5 uses 1=1	No selection means no restriction
BETWEEN 50 AND 150	Price range is inclusive

General form: **(facet1) AND (facet2 values OR-ed) AND NOT (excluded values) AND ...**.

3. Reconstructed Decision Table: Four Binary Conditions (2^4 = 16 Rules)

Conditions: C1 category selected, C2 brand selected, C3 price range set, C4 in-stock only.

Rule	C1	C2	C3	C4	Expected WHERE clause	Expected result
R1	Y	Y	Y	Y	category = ? AND brand IN (?) AND price BETWEEN ? AND ? AND in_stock = 1	Intersection of selected facets
R2	Y	Y	Y	N	category = ? AND brand IN (?) AND price BETWEEN ? AND ?	Intersection of selected facets
R3	Y	Y	N	Y	category = ? AND brand IN (?) AND in_stock = 1	Intersection of selected facets
R4	Y	Y	N	N	category = ? AND brand IN (?)	Intersection of selected facets
R5	Y	N	Y	Y	category = ? AND price BETWEEN ? AND ? AND in_stock = 1	Intersection of selected facets
R6	Y	N	Y	N	category = ? AND price BETWEEN ? AND ?	Intersection of selected facets
R7	Y	N	N	Y	category = ? AND in_stock = 1	Intersection of selected facets
R8	Y	N	N	N	category = ?	Intersection of selected facets

Rule	C1	C2	C3	C4	Expected WHERE clause	Expected result
R9	N	Y	Y	Y	brand IN (?) AND price BETWEEN ? AND ? AND in_stock = 1	Intersection of selected facets
R10	N	Y	Y	N	brand IN (?) AND price BETWEEN ? AND ?	Intersection of selected facets
R11	N	Y	N	Y	brand IN (?) AND in_stock = 1	Intersection of selected facets
R12	N	Y	N	N	brand IN (?)	Intersection of selected facets
R13	N	N	Y	Y	price BETWEEN ? AND ? AND in_stock = 1	Intersection of selected facets
R14	N	N	Y	N	price BETWEEN ? AND ?	Intersection of selected facets
R15	N	N	N	Y	in_stock = 1	Intersection of selected facets
R16	N	N	N	N	1 = 1	Full catalogue

4. Additional Rules for OR, NOT and Invalid Input

Rule	Scenario	Expected
R17	Two brands selected (Nike, Adidas)	Products of either brand (OR within facet)
R18	Brand Puma excluded	All products except Puma
R19	Nike included and Nike excluded	Empty result (include AND NOT include)
R20	Price min 150, max 50	Validation error or empty result
R21	Price min = max = 100	Products priced exactly 100
R22	Selected facets with no matching product	Empty list with message

5. Evaluation of Completeness

Aspect	Assessment
All AND combinations of four facets	Complete: 16 of 16 rules
OR inside a facet, NOT, empty result	Covered by R17 to R22
Not captured by the logs	Further facets (rating, colour), sorting, pagination, text search
Reduction possible	Rules with identical actions can merge with don't-care entries, but the full table is kept because the queries are built dynamically

Conclusion

The reconstructed model of 22 rules covers every combination of the four inferred facets plus OR, NOT and invalid-range behaviour. New facets found later extend the table by doubling the AND combinations, so the model must be re-validated against the real catalogue.